

PCGCv2 Point Cloud Geometry Compression — Reproduction Report

Reproduced work: NJUVISION/PCGCv2 (Multiscale Point Cloud Geometry Compression, DCC 2021)
Tianhao Wu | 2026-05-21

1. Goal of the Reproduction

My advisor asked me to reproduce PCGCv2, a point cloud geometry compression method (github.com/NJUVISION/PCGCv2). It is a multiscale autoencoder built on sparse convolution, published at DCC 2021.

I set the goal of this reproduction at the “reproduce the paper's metrics” level: not just getting the code to run without errors, but actually obtaining bpp, D1 PSNR and D2 PSNR on the 8iVFB dataset used in the paper, drawing the rate-distortion curves, and comparing them against the paper and the authors' official results to see whether the numbers line up.

2. Environment

PCGCv2 depends on MinkowskiEngine, a sparse-convolution library that requires an NVIDIA CUDA GPU, which my Mac does not have. So the whole reproduction was done on a rented cloud GPU. The environment I finally got working is below.

Component	Version / Configuration
Cloud platform	RunPod (pay-as-you-go GPU cloud)
GPU	NVIDIA RTX 3090, 24 GB VRAM (Ampere)
OS	Ubuntu 18.04
Container image	pytorch/pytorch:1.8.1-cuda11.1-cudnn8-devel
Python	3.8.8
CUDA	11.1 (V11.1.105)
PyTorch	1.8.1 + cu111
MinkowskiEngine	0.5.4 (compiled from source)
torchac	0.9.3
tmc3	v12 (binary bundled in the repo)
Other dependencies	h5py, pandas 1.1.5, open3d 0.13, matplotlib 3.3.4, ninja

Among these, MinkowskiEngine was installed by compiling from source, and this was the step most likely to fail. The command I used was:

```
export CUDA_HOME=/usr/local/cuda && export MAX_JOBS=4 && python setup.py install --blas=openblas --force_cuda
```

The version combination Python 3.8 / CUDA 11.1 / PyTorch 1.8.1 compiled successfully on the first try; from what I read, other combinations (especially newer CUDA versions) tend to fail.

3. Reproduction Results

I ran test.py on each of the four 8iVFB sequences (longdress, loot, redandblack, soldier). For each sequence the script automatically goes through all seven rate models r1–r7, which corresponds to seven points on one rate-distortion curve. The results for the four sequences are below.

longdress (longdress_vox10_1300)

Rate model	bpp	D1 PSNR (dB)	D2 PSNR (dB)
r1	0.024	60.89	63.65
r2	0.047	66.25	69.41
r3	0.092	69.92	72.96
r4	0.152	71.93	75.37
r5	0.246	73.60	77.52
r6	0.316	74.36	78.33
r7	0.400	75.13	79.24

loot (loot_vox10_1200)

Rate model	bpp	D1 PSNR (dB)	D2 PSNR (dB)
r1	0.024	61.66	64.80
r2	0.047	67.14	70.17
r3	0.089	70.16	73.13
r4	0.149	72.19	75.59
r5	0.242	73.89	77.67
r6	0.311	74.71	78.57
r7	0.397	75.45	79.42

redandblack (redandblack_vox10_1550)

Rate model	bpp	D1 PSNR (dB)	D2 PSNR (dB)
r1	0.026	61.33	64.63
r2	0.050	65.79	69.52
r3	0.095	68.99	72.44
r4	0.157	70.98	74.73

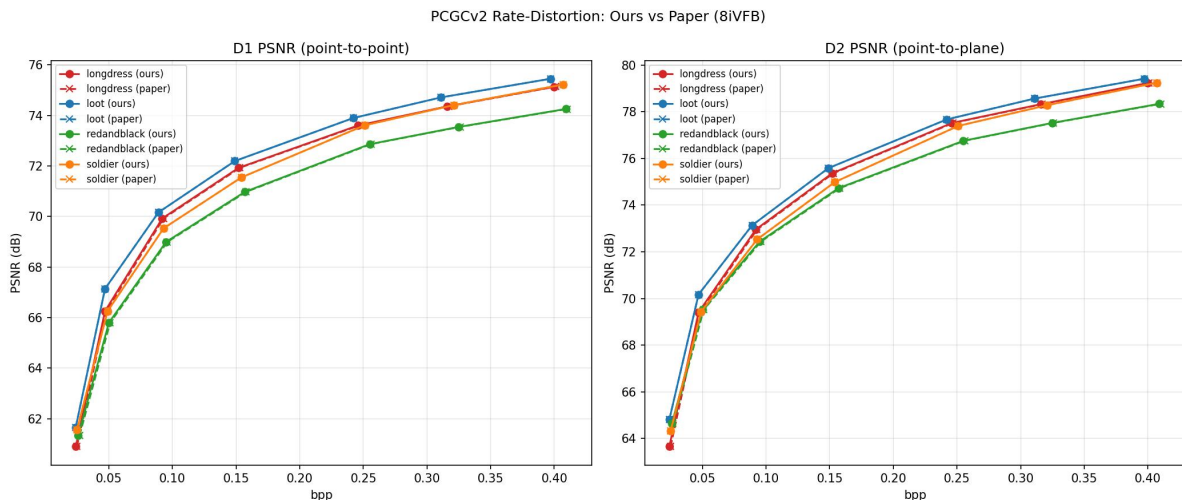
r5	0.255	72.86	76.76
r6	0.325	73.54	77.52
r7	0.409	74.25	78.34

soldier (soldier_vox10_0690)

Rate model	bpp	D1 PSNR (dB)	D2 PSNR (dB)
r1	0.025	61.57	64.31
r2	0.049	66.25	69.43
r3	0.093	69.53	72.53
r4	0.154	71.54	74.99
r5	0.251	73.61	77.39
r6	0.321	74.41	78.28
r7	0.407	75.22	79.25

Rate-Distortion Curves

I plotted the data for the four sequences as rate-distortion curves and overlaid them with the authors' official results:



Solid lines are my results; dashed lines are the authors' official results.

The two sets of curves essentially overlap and are hard to tell apart by eye. I also compared every point against the authors' official CSV files: across four sequences and seven rate points (28 points in total), the largest difference in D1/D2 PSNR from the official results is only 0.0025 dB. A difference of this magnitude most likely comes from floating-point randomness in the encoding process, so the reproduced results can be considered consistent with the paper.

4. Problems Encountered During the Reproduction

The reproduction did not go entirely smoothly; I ran into quite a few issues. I'll note the main ones roughly in the order they came up.

1. Renting a GPU was a problem right at the start. I originally planned to use AutoDL, but since I am currently outside mainland China, the real-name verification step kept failing with a “verification not available in your region” message. I switched to RunPod, an international platform that needs no real-name verification, accepts a foreign credit card directly, and requires no proxy.

2. The image and the GPU both had to be chosen carefully. RunPod's default image uses CUDA 12.4, but MinkowskiEngine needs CUDA 11.1, so I switched to a custom image, pytorch/pytorch:1.8.1-cuda11.1-cudnn8-devel. The GPU also matters — it has to be an Ampere-architecture card (I used an RTX 3090); newer cards are not supported by CUDA 11.1.

3. The cloud platform resets data on shutdown. Once a RunPod instance is stopped, the container disk is reset and only the /workspace volume is kept. So I installed all the code, data and environment under /workspace, and also wrote a rebuild.sh script that restores the environment after a restart.

4. When installing packages with apt, it reported a GPG-key-expired error for the NVIDIA software source. Since CUDA was already in the image and that source was not needed, removing it from the source list fixed apt.

5. Downloading the test data was very slow. The data is hosted on a Nanjing University network drive, and a direct connection from abroad reached only a few tens of KB/s — an estimated two-plus hours. Switching to aria2 for multi-threaded download brought it up to 3 MB/s, and it finished in about a minute.

6. Compiling MinkowskiEngine was the step I was most worried about — many people online get stuck here. Fortunately, by following the version combination strictly and installing libopenblas and ninja first, it compiled successfully on the first try.

7. The repository's scripts were still missing a few Python packages (h5py, pandas, open3d), which I installed one by one as the errors came up. For pandas I deliberately chose the older version 1.1.5 — a newer pandas would also upgrade numpy, which could affect the already-installed torch.

8. When installing open3d, it pulled in a newer matplotlib that was incompatible with numpy 1.19.2 in the environment, causing test.py to fail at the plotting step. Downgrading matplotlib back to 3.3.4 solved it.

9. test.py writes its results into the results/ directory, and this overwrote the authors' official result CSVs that came with the repository. I first backed up my own results, then used git to

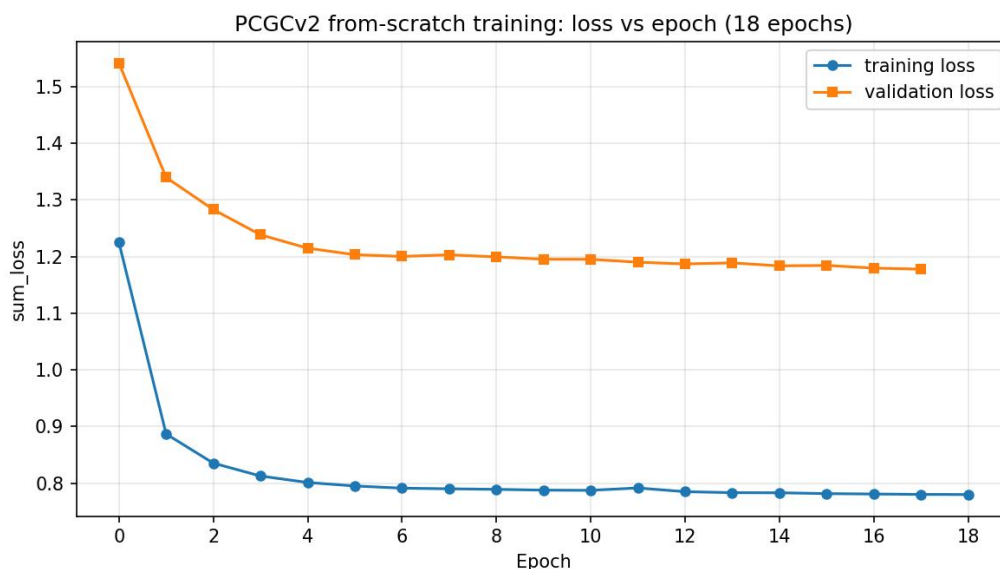
restore the authors' official CSVs, so that the numerical comparison could be done afterwards.

10. One unexpected piece of good news: I had originally planned to spend a day compiling MPEG's tmc3 tool, but it turned out the repository already includes the tmc3 and pc_error_d binaries — simply adding execute permission was enough, so this step was skipped.

5. Training from Scratch (Optional)

After finishing the metric-level reproduction, I also tried the next level — training a model from scratch with train.py. The training set is provided by the authors, about 3.7 GB, with more than 20,000 .h5 point cloud files. I trained with the default settings (alpha and beta both 1, batch_size 8) on the cloud GPU, running it in the background.

The paper trains for 50 epochs, but with a limited cloud GPU budget I stopped at epoch 18 — about 9 hours, with 19 checkpoints saved along the way. I extracted the per-epoch loss from the training log and plotted it:



Training loss and validation loss over epochs.

Both the training loss and the validation loss drop quickly in the first few epochs and then level off, with no divergence or oscillation — the training process is normal and the model is indeed converging.

Using the epoch-18 model, I encoded and decoded the longdress sequence and compared it with the authors' fully-trained model:

Item	My model (18/50 epochs)	Authors' model (full, 50 epochs)
Test sequence	longdress	longdress
bpp	0.255	0.246

D1 PSNR (dB)	73.26	73.60
--------------	-------	-------

My model, trained for only 18 of 50 epochs, has a D1 PSNR about 0.3–0.5 dB lower than the authors' fully-trained model at a similar rate. Given that the amount of training is only about one third of the paper's, this gap is small, which shows the training pipeline is correct and the model is converging normally; with the full 50 epochs it should get even closer to the paper's level.

6. Conclusion

Overall, this reproduction was successful. Across the four 8iVFB sequences and seven rate points, the bpp and D1/D2 PSNR I obtained are within 0.0025 dB of the authors' official results, and the rate-distortion curves essentially overlap with the paper.

The most critical step in the whole process was compiling MinkowskiEngine, where the versions must match exactly. Most of the other pitfalls were related to my overseas network environment, the cloud platform's shutdown behavior, and dependency versions — all recorded above. The from-scratch training also ran successfully, with the model I trained myself showing only a small gap from the paper's metrics.